

DATABASE SYSTEMS

DB Lecture 2

The Relational Model and SQL

Topics covered in this lecture:

- The relational model: relations, tuples, attributes, domains
- Keys: super key, candidate key, primary key, foreign key
- Integrity constraints: entity, referential, domain
- Relational algebra operations
- SQL Data Definition Language (DDL)
- SQL Data Manipulation Language (DML)
- Joins and set operations
- Views, aggregation, and grouping

Database Systems Lecture Series
Tables, Keys, Constraints, and the Structured Query Language

1. The Relational Model

Proposed by E. F. Codd in 1970, the **relational model** represents data as a collection of two-dimensional tables called **relations**. Its simplicity and strong mathematical foundation (based on set theory and predicate logic) made it the dominant data model for decades, and it remains the backbone of most enterprise applications today.

1.1 Basic Terminology

Formal Term	Informal Term	Meaning
Relation	Table	A set of tuples sharing the same attributes.
Tuple	Row / Record	A single ordered set of attribute values.
Attribute	Column / Field	A named property of a relation.
Domain	Data type / value set	The set of legal values an attribute may take.
Degree	Number of columns	The number of attributes in a relation.
Cardinality	Number of rows	The number of tuples in a relation, at a given point in time.

1.2 Properties of Relations

- Each cell holds a single, atomic value (no repeating groups) — this is the essence of **first normal form**, discussed further in DB Lecture 3.
- All values in a column come from the same domain.
- Each row is distinct; no two tuples in a relation are identical.
- The order of rows and columns is not significant to the meaning of the relation.
- Each attribute has a unique name within its relation.

2. Keys in the Relational Model

Keys are attributes (or combinations of attributes) used to uniquely identify tuples in a relation and to establish relationships between relations.

2.1 Types of Keys

- **Super key:** any set of attributes that uniquely identifies a tuple. A relation may have many super keys.
- **Candidate key:** a minimal super key — no proper subset of it is also a super key. A relation may have several candidate keys.
- **Primary key:** the candidate key chosen by the database designer to be the principal means of identifying tuples. Only one candidate key can serve as the primary key.
- **Alternate key:** any candidate key that was not chosen as the primary key.
- **Composite key:** a candidate or primary key composed of two or more attributes.
- **Foreign key:** an attribute (or set of attributes) in one relation that references the primary key of another (or the same) relation, used to represent relationships between tuples in different relations.

2.2 Example

Consider two relations: **Student(StudentID, Name, DeptID)** and **Department(DeptID, DeptName)**. In Student, StudentID is the primary key. DeptID in Student is a foreign key referencing DeptID, the primary key of Department. This foreign key enforces that every student must belong to a department that actually exists.

```
Department Student
+-----+-----+ +-----+-----+
| DeptID | DeptName | | StudentID | Name | DeptID |
+-----+-----+ +-----+-----+
| D01 | CS | | S100 | Ayesha | D01 |
| D02 | Math | | S101 | Bilal | D02 |
+-----+-----+ +-----+-----+
```

3. Integrity Constraints

Integrity constraints are rules that the DBMS enforces automatically to keep the database accurate and consistent as data changes.

3.1 Domain Constraint

Every attribute value must be drawn from the attribute's declared domain (data type and any additional restriction, such as a value range or an enumerated list of allowed values).

3.2 Entity Integrity Constraint

No attribute participating in the primary key of a relation may be NULL. This guarantees that every tuple can be uniquely identified.

3.3 Referential Integrity Constraint

A foreign key value must either match an existing primary key value in the referenced relation, or be entirely NULL (if NULLs are permitted). This prevents "dangling references" — for example, a Student row that points to a DeptID that does not exist in the Department table.

3.4 Referential Actions

When a referenced row is deleted or its key updated, the DBMS must decide what happens to dependent rows. Common referential actions specified in SQL include:

Action	Effect
CASCADE	Automatically delete/update the dependent rows as well.
SET NULL	Set the foreign key in dependent rows to NULL.
RESTRICT / NO ACTION	Reject the delete/update if dependent rows exist.
SET DEFAULT	Set the foreign key in dependent rows to a default value.

4. Relational Algebra

Relational algebra is a procedural query language: a set of operations that take one or more relations as input and produce a new relation as output. It provides the formal, mathematical foundation on which SQL query processing is built.

4.1 Basic (Unary) Operations

- **Selection (σ):** selects tuples that satisfy a given condition, e.g. $\sigma_{\text{Age} > 20}(\text{Student})$.
- **Projection (π):** selects specific attributes (columns), discarding duplicates, e.g. $\pi_{\text{Name, DeptID}}(\text{Student})$.
- **Rename (ρ):** renames a relation or its attributes.

4.2 Set-Based (Binary) Operations

- **Union ($\cup$):** combines tuples from two union-compatible relations, removing duplicates.
- **Intersection ($\cap$):** returns tuples that appear in both relations.
- **Set difference ($-$):** returns tuples in the first relation but not the second.
- **Cartesian product ($\times$):** combines every tuple of one relation with every tuple of another.

4.3 Derived Operations

- **Join ($\bowtie$):** combines related tuples from two relations based on a common attribute, most commonly an **equi-join** comparing attributes for equality; can be expressed as a selection over a Cartesian product.
- **Division ($\div$):** used for queries of the form "find X that are related to all Y", e.g. "find students enrolled in every course offered."

SQL, which we turn to next, is a declarative language: rather than specifying the sequence of algebraic operations to perform, the user simply states what result is wanted, and the DBMS's query optimizer decides how to translate that into an efficient sequence of relational-algebra-like operations internally.

5. Introduction to SQL

SQL (Structured Query Language) is the standard language for defining and manipulating relational databases. It combines DDL, DML, DCL, and TCL capabilities in a single, largely declarative language.

5.1 SQL Data Types (Common Examples)

Type	Description
INT / INTEGER	Whole numbers.
DECIMAL(p,s) / NUMERIC	Fixed-point numbers with precision p and scale s.
VARCHAR(n)	Variable-length character string, up to n characters.
CHAR(n)	Fixed-length character string of exactly n characters.
DATE / TIME / TIMESTAMP	Date and/or time values.
BOOLEAN	True/false values (supported natively in some DBMSs).

6. SQL Data Definition Language (DDL)

6.1 CREATE TABLE

The CREATE TABLE statement defines a new relation, its attributes, their data types, and any constraints.

```
CREATE TABLE Department (  
  DeptID CHAR(4) PRIMARY KEY,  
  DeptName VARCHAR(50) NOT NULL UNIQUE  
);  
  
CREATE TABLE Student (  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50) NOT NULL,  
  DeptID CHAR(4),  
  GPA DECIMAL(3,2) CHECK (GPA >= 0 AND GPA <= 4.0),  
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
  ON DELETE SET NULL  
);
```

6.2 ALTER TABLE and DROP TABLE

```
ALTER TABLE Student ADD COLUMN Email VARCHAR(100);  
ALTER TABLE Student DROP COLUMN Email;  
DROP TABLE Student;
```

6.3 Common Constraints

- **PRIMARY KEY** — uniquely identifies each row; implies NOT NULL and UNIQUE.
- **FOREIGN KEY ... REFERENCES** — enforces referential integrity to another table.
- **NOT NULL** — disallows missing values for an attribute.
- **UNIQUE** — disallows duplicate values for an attribute (NULLs are typically still allowed).
- **CHECK** — enforces an arbitrary boolean condition on attribute values.
- **DEFAULT** — supplies a default value when none is specified on insert.

7. SQL Data Manipulation Language (DML)

7.1 INSERT, UPDATE, DELETE

```
INSERT INTO Department (DeptID, DeptName) VALUES ('D01', 'Computer Science');
```

```
UPDATE Student SET GPA = 3.8 WHERE StudentID = 100;
```

```
DELETE FROM Student WHERE StudentID = 100;
```

7.2 The SELECT Statement

SELECT is the core of SQL's data retrieval capability. Its general form follows the order: SELECT (columns), FROM (tables), WHERE (row filter), GROUP BY (grouping), HAVING (group filter), ORDER BY (sort).

```
SELECT Name, GPA
FROM Student
WHERE DeptID = 'D01' AND GPA > 3.0
ORDER BY GPA DESC;
```

7.3 Filtering with WHERE

- Comparison operators: =, <>, <, >, <=, >=
- Logical operators: AND, OR, NOT
- BETWEEN ... AND ... for ranges
- IN (...) for membership in a list
- LIKE for pattern matching, e.g. Name LIKE 'A%'
- IS NULL / IS NOT NULL to test for missing values

8. Joins

Joins combine rows from two or more tables based on a related column, typically a foreign-key/primary-key pair. Joins are one of the most important and most frequently used features of SQL.

8.1 INNER JOIN

Returns only rows that have matching values in both tables.

```
SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d ON s.DeptID = d.DeptID;
```

8.2 OUTER JOINS

- **LEFT OUTER JOIN:** returns all rows from the left table, with NULLs for unmatched columns from the right table.
- **RIGHT OUTER JOIN:** returns all rows from the right table, with NULLs for unmatched columns from the left table.
- **FULL OUTER JOIN:** returns all rows from both tables, matching where possible and filling with NULLs elsewhere.

```
SELECT s.Name, d.DeptName
FROM Student s
LEFT OUTER JOIN Department d ON s.DeptID = d.DeptID;
-- Includes students even if their DeptID is NULL or unmatched
```

8.3 Self Joins and Cross Joins

A **self join** joins a table to itself, useful for hierarchical data such as an employee-manager relationship stored in a single table. A **CROSS JOIN** produces the Cartesian product of two tables and should be used deliberately, since it can produce very large result sets.

9. Aggregation and Grouping

9.1 Aggregate Functions

Function	Purpose
COUNT(*)	Number of rows.
SUM(col)	Total of numeric values in a column.
AVG(col)	Average of numeric values in a column.
MIN(col) / MAX(col)	Smallest / largest value in a column.

9.2 GROUP BY and HAVING

GROUP BY partitions rows into groups sharing the same value(s) in specified columns, so that aggregate functions are computed per group rather than over the whole table. HAVING filters groups after aggregation, analogous to how WHERE filters individual rows before aggregation.

```
SELECT DeptID, COUNT(*) AS NumStudents, AVG(GPA) AS AvgGPA
FROM Student
GROUP BY DeptID
HAVING COUNT(*) > 5
ORDER BY AvgGPA DESC;
```

9.3 Subqueries

A subquery is a query nested inside another query, often used in the WHERE or FROM clause. Subqueries can be correlated (referencing the outer query) or uncorrelated (independent).

```
SELECT Name FROM Student
WHERE GPA > (SELECT AVG(GPA) FROM Student);
```

10. Views

A **view** is a virtual table defined by a stored SQL query. It does not store data itself (in the simple case); instead, its result is computed on demand whenever the view is queried.

```
CREATE VIEW HighAchievers AS
SELECT StudentID, Name, GPA
FROM Student
WHERE GPA >= 3.5;

SELECT * FROM HighAchievers ORDER BY GPA DESC;
```

10.1 Why Use Views?

- **Simplify complex queries:** hide joins and filters behind a simple name.
- **Security:** expose only specific columns or rows to certain users, without granting access to the underlying base tables.
- **Logical data independence:** insulate applications from changes in the underlying table structure, as long as the view's output stays the same.

Some views are **updatable**, meaning INSERT/UPDATE/DELETE on the view is translated into corresponding changes on the base tables, though this is only possible for views that map cleanly back to a single underlying table without aggregation or certain joins.

11. Set Operations in SQL

SQL provides direct equivalents of the relational-algebra set operations, applied to the results of two SELECT statements that must be union-compatible (same number of columns, compatible types).

```
SELECT Name FROM Student WHERE DeptID = 'D01'  
UNION  
SELECT Name FROM Student WHERE GPA > 3.5;
```

```
SELECT Name FROM Student WHERE DeptID = 'D01'  
INTERSECT  
SELECT Name FROM Student WHERE GPA > 3.5;
```

```
SELECT Name FROM Student WHERE DeptID = 'D01'  
EXCEPT  
SELECT Name FROM Student WHERE GPA > 3.5;
```

UNION removes duplicate rows from the combined result by default; UNION ALL keeps duplicates and is generally faster since it skips the duplicate-elimination step.

12. Summary

This lecture covered the relational model in depth: how data is represented as relations made up of tuples and attributes, how keys uniquely identify tuples and connect relations together, and how integrity constraints keep the data valid over time. We introduced relational algebra as the theoretical foundation for query processing, and then covered SQL — the practical, industry-standard language for defining schemas (DDL), manipulating data (DML), joining tables, aggregating results, and creating views.

Key Takeaways

- A relation is a set of tuples; primary keys uniquely identify tuples, and foreign keys connect relations.
- Entity integrity and referential integrity are the two fundamental integrity constraints of the relational model.
- Relational algebra operations (selection, projection, join, etc.) underlie how SQL queries are actually executed.
- SQL's SELECT statement, combined with joins, GROUP BY, and subqueries, can express a very wide range of queries declaratively.

Looking Ahead

DB Lecture 3 addresses **database design and normalization**: how to design a relational schema that avoids redundancy and update anomalies, using functional dependencies and the normal forms (1NF through BCNF).

13. Review Questions

- 1 Define relation, tuple, attribute, and domain, and explain how they correspond to the informal terms table, row, column, and data type.
- 2 Explain the difference between a candidate key, a primary key, and a foreign key, using an original example with two related tables.
- 3 What is the difference between entity integrity and referential integrity? Give an example of each constraint being violated.
- 4 List the referential actions available for foreign keys (CASCADE, SET NULL, RESTRICT, SET DEFAULT) and describe a real scenario where each would be appropriate.
- 5 Write a SELECT statement that joins two tables of your choosing and filters, groups, and orders the result.
- 6 Explain the difference between an INNER JOIN and a LEFT OUTER JOIN, and describe a situation where the results would differ.
- 7 What is a view, and why might a database designer create one instead of granting direct access to base tables?
- 8 Explain the difference between UNION and UNION ALL in SQL.

14. Glossary of Key Terms

Term	Definition
Relation	A table consisting of rows (tuples) and columns (attributes).
Primary key	The chosen candidate key that uniquely identifies each tuple.
Foreign key	An attribute referencing the primary key of another relation.
Referential integrity	The rule that foreign key values must match an existing primary key or be NULL.
Join	An operation combining rows from two relations based on a related column.
View	A virtual table defined by a stored query.
Aggregate function	A function (COUNT, SUM, AVG, MIN, MAX) that computes a single value from a set of rows.
Subquery	A query nested inside another SQL query.

This glossary summarizes the core SQL and relational-model vocabulary used throughout DB Lecture 2. A firm grasp of these terms is essential before moving on to the design and normalization concepts in DB Lecture 3.